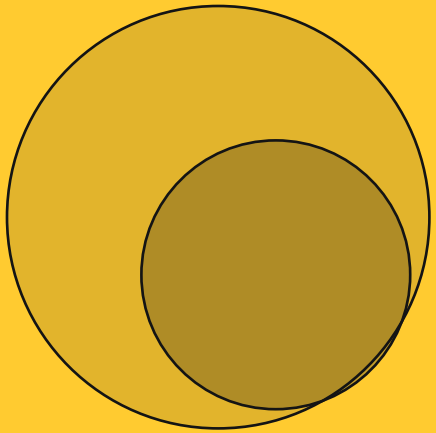
Aufbau einer Datenbank

aus nicht-normalisierten Daten

Tools:

pgAdmin 4 · DBeaver · jupyter · pandas · SQLAlchemy

SQL · Python · PostgreSQL



Agenda

1. Normalisieren des Excel-Tabellenblatts
2. Tabellen in Postgres importieren
3. Rollen und Benutzer anlegen
4. Ansichten für die Datensicherheit erstellen
5. Einsatz eines Datenanalysten
6. Datenzugriffsprotokollierung implementieren
7. Einschränkung des Datenzugriffs (Python)
8. Erweiterte Analyse — Business-Insights

1 Normalisieren des Excel-Tabellenblatts

Ausgangstabelle — Probleme

KundeName

Vor- & Nachname in einem Feld

KundeAdresse

Straße, PLZ, Ort in einem Feld

Kategorie

Wiederholt sich bei jedem Produkt

ProduktPreis

Redundant bei jeder Bestellung

KreditkarteNr

Sensible Daten ungeschützt

→ Normalform

prodkategorie

ProdkategorieID · Kategorie

produkte

ProduktID · ProduktName · ProduktPreis · ProdkategorieID

kunden

KundenID · Vorname · Nachname · Strasse · PLZ · Ort · Kreditkarte

verkaeufe

BestellID · Menge · Bestelldatum · KundenID · ProduktID

2 Tabellen in PostgreSQL importieren

1

pgAdmin 4

Neue leere Datenbank anlegen: mein_praxisprojekt

2

DBeaver

Verbindung aufbauen: localhost, Port 5433

3

DBeaver SQL

Tabellen per CREATE TABLE mit Primär- & Fremdschlüsseln anlegen

4

Jupyter

CSV-Daten via pandas.to_sql() in die DB einfügen

Import via: SQLAlchemy pandas

3 Rollen und Benutzer anlegen

DBeaver

ROLLE

callcenter_mitarbeiter

BENUTZER

mitarbeiter

BERECHTIGUNGEN

SELECT auf kunden_view_sicher (nur View, kein direkter Tabellenzugriff)

```
CREATE ROLE callcenter_mitarbeiter;
GRANT SELECT ON kunden_view_sicher
  TO callcenter_mitarbeiter;
CREATE USER mitarbeiter PASSWORD '...';
-- Passwort nie im Code → .env verwenden
GRANT callcenter_mitarbeiter TO mitarbeiter;
```

ROLLE

datenanalyst

BENUTZER

schmidt

BERECHTIGUNGEN

SELECT auf verkaeufe, produkte, prodkategorie, kunden

```
CREATE ROLE datenanalyst;
GRANT SELECT ON verkaeufe, produkte,
  prodkategorie, kunden TO datenanalyst;
CREATE USER schmidt PASSWORD '...';
GRANT datenanalyst TO schmidt;
```

4 Ansichten für die Datensicherheit

DBeaver

1

Zugriff entziehen

REVOKE ALL ON kunden FROM PUBLIC;
Direktszugriff für alle gesperrt — nur noch über View möglich.

```
REVOKE ALL ON kunden FROM PUBLIC;
```

2

Sicheren View erstellen

Kreditkartennummer maskiert: nur letzte 3 Ziffern sichtbar.

```
CREATE VIEW kunden_view_sicher AS
  SELECT "KundenID", "KundeVorname",
"KundeNachname",
    '************ ' || RIGHT("KundeKreditkarte",
3)
      AS Kreditkarte_letzte3
  FROM kunden;
```

3

Zugriff erteilen

Nur callcenter_mitarbeiter darf den View abfragen.

```
GRANT SELECT ON kunden_view_sicher
  TO callcenter_mitarbeiter;
```

5 Einsatz eines Datenanalysten

Verbindung zur DB als Benutzer schmidt (Rolle: datenanalyst) · nur Lesezugriff · via SQLAlchemy

121.378 €
Umsatz Top-Kunde
Linnet Firmage

Mittwoch
Häufigster
Einkaufstag (21×)

32 Mon.
Datenzeitraum
Okt 2020 – Jun 2023

sales_analysis.ipynb

```
df_merged = df_verkaeufe.merge(df_produkte, on="ProduktID") \
                 .merge(df_kunden, on="KundenID")

# Top-Kunde
top = df_merged.groupby("KundenID")["Gesamtpreis"].sum().idxmax()

# Zeitraum
start = df_verkaeufe["Bestelldatum"].min()  # 2020-10-01
end   = df_verkaeufe["Bestelldatum"].max()  # 2023-06-30

# Häufigster Wochentag
df["Wochentag"] = df["Bestelldatum"].dt.day_name()
```

6 Datenzugriffsprotokollierung — Trigger

DBeaver

Was protokolliert wird:

benutzername

current_user — wer die Aktion ausführt

tabelle

TG_TABLE_NAME — welche Tabelle

zugriffsart

TG_OP — INSERT / UPDATE / DELETE

zugriffszeit

NOW() — Zeitstempel

Trigger protokolliert INSERT, UPDATE, DELETE — für SELECT-Audit wäre «pgaudit» erforderlich

database_setup.sql

```
-- Protokolltabelle
CREATE TABLE zugriffsprotokoll (
  id            SERIAL PRIMARY KEY,
  benutzername  TEXT,
  tabelle       TEXT,
  zugriffsart   TEXT,
  zugriffszeit  TIMESTAMP DEFAULT NOW()
);

-- Trigger-Funktion
CREATE OR REPLACE FUNCTION log_zugriff()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO zugriffsprotokoll
    (benutzername, tabelle, zugriffsart)
  VALUES
    (current_user, TG_TABLE_NAME, TG_OP);
  RETURN NULL;
END;
$$ LANGUAGE plpgsql;

-- Trigger anhängen
CREATE TRIGGER kunden_zugriff_log
AFTER INSERT OR UPDATE OR DELETE ON kunden
FOR EACH STATEMENT
EXECUTE FUNCTION log_zugriff();

-- Test
SET ROLE mitarbeiter;
UPDATE kunden SET "KundeVorname" = 'Alla'
  WHERE "KundenID" = 1;
```

7 Einschränkung des Datenzugriffs für Python

Prinzip

Python-Analyse läuft ausschließlich
als Benutzer schmidt
mit der Rolle datenanalyst.

```
SELECT * FROM kunden;
```

✓ **Erlaubt — Lesezugriff**

```
UPDATE kunden SET "KundeVorname" = 'Test'
WHERE "KundenID" = 1;
```

✗ **Abgelehnt — kein Schreibrecht**

sales_analysis.ipynb

```
from sqlalchemy import create_engine
from dotenv import load_dotenv
import os

load_dotenv()

# Verbindung als schmidt (Rolle: datenanalyst)
engine_analyst = create_engine(
    f"postgresql+psycopg2://"
    f"{os.environ['DB_USER_ANALYST']}:"
    f"{os.environ['DB_PASSWORD_ANALYST']}"
    f"@{os.environ['DB_HOST']}:"
    f"{os.environ['DB_PORT']}/"
    f"{os.environ['DB_NAME']}"
)

# Aktuellen Benutzer prüfen
df_user = pd.read_sql(
    "SELECT current_user, current_role;",
    engine_analyst
)
print(df_user)
# → current_user: schmidt | current_role: datenanalyst

# SELECT funktioniert ✓
df = pd.read_sql("SELECT * FROM kunden;",
                 engine_analyst)

# UPDATE wird abgelehnt ✗
# engine_analyst.execute(...)  → Permission denied
```

8 Erweiterte Analyse — Business-Insights

SQL: JOIN · GROUP BY · HAVING · CASE WHEN · RANK() OVER · SUM() OVER

123.680 €

Uhren-Umsatz

31% des Gesamtumsatzes
GROUP BY + SUM()

120.000 €

Stärkster Monat

Oktober 2020
DATE_TRUNC('month', ...)

6 / 30

Einmalkäufer

20% der Kunden
CASE WHEN COUNT = 1

RANK()

Top-Produkt je Kategorie

Rolex: Uhren
RANK() OVER (PARTITION BY)

37.200 €

Ø Premium-Kunde

Kunden über Durchschnitt
HAVING AVG(...) > Subquery

75%

Top-3 Kategorien

Uhren + Schmuck + Möbel
SUM() OVER () ohne PARTITION

Fazit

Was dieses Projekt beweist:

- SQL für Struktur, Schlüssel & Zugriffskontrolle
- pandas für effizienten Import & Analyse
- Rollenkonzept schützt sensible Daten
- Window Functions für komplexe Insights
- Passwörter sicher via .env — nie im Code

Verwendete Tools

pgAdmin 4

DBeaver

Jupyter

pandas

SQLAlchemy

Excel

Vielen Dank für Ihre Aufmerksamkeit